

TutioLMS — Site Functions & Developer Guide

Complete reference for how the project is structured, how each feature works, and how to run both the frontend and the planned NestJS backend.

Table of Contents

- 1. [Project Overview](#)
- 2. [Technology Stack](#)
- 3. [How to Start the Project](#)
- 4. [Project Structure](#)
- 5. [Frontend Architecture](#)
- 6. [Public Site — Pages & Features](#)
- 7. [Admin Dashboard — Pages & Features](#)
- 8. [Authentication & Role System](#)
- 9. [Data Layer](#)
- 10. [Theming & Styling System](#)
- 11. [Reusable Component Library](#)
- 12. [NestJS Backend Reference](#)
- 13. [Database Schema \(Prisma\)](#)
- 14. [API Endpoints Reference](#)
- 15. [Configuration Files](#)
- 16. [Environment Variables](#)
- 17. [Adding New Features](#)

1. Project Overview

TutioLMS is a tuition management system for Mr. Kamal Perera — a mathematics and physics teacher with 3 learning centres across Sri Lanka (Colombo, Kandy, Gampaha).

The system has two layers:

Layer	Technology	Status
Public-facing website	Next.js 14 (App Router)	Live — fully functional
Admin dashboard	Next.js 14 (same app, <code>/admin</code> route)	Live — uses mock data
Backend API	NestJS + Prisma + PostgreSQL	Reference code only — not yet integrated

Current stage: The admin dashboard uses local React state and `localStorage` to simulate API calls. The `backend-reference/` folder contains the production-ready NestJS architecture to wire up when a real server is needed.

2. Technology Stack

Frontend (Active)

Tool	Version	Purpose
Next.js	14.2.14	React framework, App Router, SSG
React	18	UI rendering
TypeScript	5	Type safety across all files
Tailwind CSS	3.4.1	Utility-first styling
Lucide React	0.441.0	Icon library (SVG icons)
react-colorful	5.6.1	HSL color picker for Theme Editor

Backend Reference (Not Yet Integrated)

Tool	Purpose
NestJS	Server framework (modules, controllers, guards)
Prisma ORM	Database access + schema management
PostgreSQL	Primary relational database
passport-jwt	JWT extraction from Authorization header
bcrypt	Password hashing
@nestjs/config	Environment variable management

3. How to Start the Project

Frontend (Next.js)

Prerequisites:

- Node.js 18 or higher
- npm (comes with Node.js)

Step 1 — Install dependencies

```
cd c:/Users/AVONET/Desktop/lms
npm install
```

Step 2 — Run development server

```
npm run dev
```

The site starts at **http://localhost:3000**

Step 3 — Open in browser

- Public site: <http://localhost:3000>
- Admin login: <http://localhost:3000/admin>

Other available scripts:

Command	What it does
<code>npm run dev</code>	Start development server with hot reload
<code>npm run build</code>	Build optimised production bundle
<code>npm run start</code>	Start production server (after build)
<code>npm run lint</code>	Run ESLint on all source files

Backend (NestJS — Setup Guide for Production)

The backend is provided as reference code in [backend-reference/](#). Follow these steps to set it up as a real server:

Step 1 — Create a new NestJS project

```
npm install -g @nestjs/cli
nest new tutiolms-api
cd tutiolms-api
```

Step 2 — Install required packages

```
npm install @nestjs/passport @nestjs/jwt passport passport-jwt bcrypt
@nestjs/config
npm install -D @types/passport-jwt @types/bcrypt
npm install @prisma/client prisma
```

Step 3 — Copy backend-reference files Copy all files from [backend-reference/](#) into your new NestJS project:

```
backend-reference/prisma/schema.prisma → prisma/schema.prisma
backend-reference/src/auth/           → src/auth/
backend-reference/src/banners/         → src/banners/
backend-reference/src/learning-materials/ → src/learning-materials/
backend-reference/src/site-settings/   → src/site-settings/
```

Step 4 — Create a `.env` file in the NestJS project root:

```

DATABASE_URL="postgresql://username:password@localhost:5432/tutiolms"
JWT_SECRET="your-secret-key-minimum-32-chars"
JWT_EXPIRES_IN="7d"
PORT=4000

```

Step 5 — Set up the database

```

# Create database tables from schema
npx prisma migrate dev --name init

# Seed initial admin user
npx prisma db seed

```

Step 6 — Start the API server

```
npm run start:dev
```

API server starts at **http://localhost:4000**

Step 7 — Connect frontend to API In the Next.js project, replace mock data imports with real `fetch()` calls to `http://localhost:4000`.

4. Project Structure

```

c:/Users/AVONET/Desktop/lms/
├── src/
│   ├── app/                                # Next.js App Router pages
│   │   ├── layout.tsx                      # Root HTML layout (metadata, PublicChrome)
│   │   ├── globals.css                    # CSS variables, Tailwind base, typography
│   │   ├── page.tsx                       # Homepage (/)
│   │   ├── not-found.tsx                  # 404 error page
│   │   ├── classes/page.tsx              # /classes
│   │   ├── institutions/page.tsx         # /institutions
│   │   ├── learning-centre/page.tsx      # /learning-centre
│   │   ├── contact/page.tsx             # /contact
│   │   └── admin/                         # Admin dashboard (all /admin/* routes)
│   │       ├── layout.tsx                # Admin shell (sidebar + header)
│   │       ├── page.tsx                  # /admin – login page
│   │       ├── dashboard/page.tsx        # /admin/dashboard
│   │       ├── banners/page.tsx          # /admin/banners
│   │       ├── institutions/page.tsx     # /admin/institutions
│   │       ├── theme/page.tsx            # /admin/theme
│   │       ├── teachers/page.tsx         # /admin/teachers
│   │       └── learning-centre/page.tsx  # /admin/learning-centre

```

└─ profile/page.tsx	# /admin/profile
└─ components/	# All React components
└─ layout/	
└─ Navbar.tsx	# Public sticky navigation bar
└─ Footer.tsx	# Public footer (3 columns)
└─ PublicChrome.tsx	# Conditionally shows/hides Navbar+Footer
└─ home/	
└─ HeroCarousel.tsx	# Auto-rotating hero slider
└─ AboutSection.tsx	# About teacher + stats section
└─ classes/	
└─ AccordionList.tsx	# Renders all 5 class category accordions
└─ AccordionItem.tsx	# Individual expandable accordion card
└─ institutions/	
└─ InstitutionCard.tsx	# Location card with timetable
└─ learning-centre/	
└─ DocumentCard.tsx	# Downloadable document card
└─ contact/	
└─ PersonalContactCard.tsx	# Teacher profile + social links
└─ InstitutionContactCard.tsx	# Branch contact info card
└─ ui/	# Shared primitive components
└─ Button.tsx	# Button (primary/outline/ghost, 3 sizes)
└─ Card.tsx	# White card wrapper
└─ Badge.tsx	# Label badge
└─ SectionHeading.tsx	# Page section title with underline
└─ admin/	# Admin-only components
└─ layout/	
└─ AdminSidebar.tsx	# Role-filtered nav + logout
└─ AdminHeader.tsx	# Top bar with breadcrumb + user chip
└─ shared/	
└─ DataTable.tsx	# Generic typed data table
└─ Modal.tsx	# Portal modal with Escape-to-close
└─ ProtectedRoute.tsx	# Role-based route guard
└─ PageHeader.tsx	# Page title + action button row
└─ StatCard.tsx	# Stat KPI card with icon
└─ ConfirmDialog.tsx	# Delete confirmation modal
└─ banners/BannerForm.tsx	
└─ institutions/InstitutionForm.tsx	
└─ teachers/TeacherForm.tsx	
└─ learning-centre/MaterialForm.tsx	
└─ profile/ProfileForm.tsx	
└─ theme/ThemeEditor.tsx	
└─ contexts/	
└─ AuthContext.tsx	# React context for auth state + mock login
└─ data/	# Static data (replaces database for now)
└─ navigation.ts	# Public nav menu items
└─ slides.ts	# Hero carousel slide content
└─ classes.ts	# All 5 class categories with schedules
└─ institutions.ts	# 3 learning centre locations + timetables
└─ documents.ts	# 9 downloadable learning materials
└─ contact.ts	# Teacher + institution contact details
└─ admin/	# Mock data for admin dashboard

```

├── mockUsers.ts           # Login credentials + user records
├── mockBanners.ts         # 3 sample carousel banners
├── mockTeachers.ts        # 2 teacher records
├── mockMaterials.ts        # 8 learning materials
├── mockSettings.ts         # Default theme + nav visibility settings
├── types/
│   ├── index.ts           # Public site TypeScript interfaces
│   └── admin.ts           # Admin dashboard TypeScript interfaces
├── lib/
│   └── icons.ts           # Lucide icon name mappings
├── backend-reference/      # NestJS backend (reference – not integrated)
│   ├── prisma/schema.prisma # PostgreSQL schema (6 models)
│   └── src/
│       ├── auth/          # JWT auth + RBAC system
│       │   ├── roles.enum.ts
│       │   ├── roles.decorator.ts
│       │   ├── roles.guard.ts
│       │   ├── jwt.strategy.ts
│       │   └── auth.service.ts
│       ├── banners/banners.controller.ts
│       ├── learning-materials/learning-materials.controller.ts
│       └── site-settings/site-settings.controller.ts
├── package.json
├── tsconfig.json          # TypeScript config (excludes backend-
reference)
├── tailwind.config.ts     # Custom color system + font config
├── next.config.mjs        # Image optimization disabled (unoptimized:
true)
└── postcss.config.js

```

5. Frontend Architecture

How Next.js App Router Works Here

Every folder inside `src/app/` that contains a `page.tsx` becomes a URL route automatically:

File	URL
<code>src/app/page.tsx</code>	<code>/</code>
<code>src/app/classes/page.tsx</code>	<code>/classes</code>
<code>src/app/admin/page.tsx</code>	<code>/admin</code>
<code>src/app/admin/banners/page.tsx</code>	<code>/admin/banners</code>

Layouts wrap child pages. There are two layouts:

1. `src/app/layout.tsx` — Root layout. Applies to every page. Wraps children in `<PublicChrome>`.
2. `src/app/admin/layout.tsx` — Admin layout. Applies only to `/admin/**` pages. Provides the sidebar + header shell.

How PublicChrome Works

`PublicChrome` (`src/components/layout/PublicChrome.tsx`) is a "use client" component that reads the current URL with `usePathname()`. If the URL starts with `/admin`, it renders the children directly with no Navbar or Footer. Otherwise it wraps children with the public `<Navbar>` and `<Footer>`.

This is the only mechanism needed to completely separate public site chrome from the admin dashboard.

Server vs Client Components

- **Server components** (default): pages that only render static HTML — e.g. `src/app/classes/page.tsx`
- **Client components** (marked "use client"): any component using `useState`, `useEffect`, `useRouter`, or browser APIs — e.g. all admin pages, `HeroCarousel`, `Navbar`, `AuthContext`

Import Alias

All imports use `@/` as a shortcut for `src/`:

```
import { useAuth } from "@/contexts/AuthContext"; // = src/contexts/AuthContext
import type { Banner } from "@/types/admin"; // = src/types/admin
import { mockBanners } from "@/data/admin/mockBanners"; // = src/data/admin/mockBanners
```

6. Public Site — Pages & Features

Home Page (/)

Components used: `HeroCarousel`, `AboutSection`

HeroCarousel features:

- Displays 4 rotating slides (data from `src/data/slides.ts`)
- Auto-advances every 5 seconds
- Manual navigation with left/right arrows
- Dot indicators at the bottom
- Pauses on hover
- Each slide has a heading, subheading, background colour, and optional CTA button

AboutSection features:

- Teacher profile image (left column)
- 4 qualifications listed
- 4 stat cards: 800+ students, 15 years experience, 95% pass rate, Top Teacher Award

- CTA button linking to </contact>

Classes Page (</classes>)

Component: [AccordionList](#) → [AccordionItem](#)

Data source: [src/data/classes.ts](#)

Displays 5 class categories in expandable accordion cards:

Category	Capacity	Example Schedule
Hall Classes	80–100 students	Saturday 8:00 AM – 11:00 AM
Group Classes	10–15 students	Mon/Wed/Fri 4:00 PM – 6:00 PM
Paper Classes	Past paper practice	Sunday 9:00 AM – 11:30 AM
Revision Classes	Intensive prep	Before exam season
Online Classes	Zoom-based	Flexible scheduling

Each accordion card shows:

- Category name + icon + description
- Full schedule table (Subject, Level, Day, Time, Fee, Venue)
- Special notes section (e.g. "Limited seats — register early")

Institutions Page (</institutions>)

Component: [InstitutionCard](#)

Data source: [src/data/institutions.ts](#)

3 learning centres displayed as cards:

Institution	Location	Weekly Classes
Colombo Learning Hub	45/B, Galle Road, Colombo 03	6 sessions
Kandy Education Centre	12, Peradeniya Road, Kandy	5 sessions
Gampaha Study Circle	78, Yakkala Road, Gampaha	5 sessions

Each card shows name, address, phone, map link, and embedded timetable with Day/Time/Subject/Level columns.

Learning Centre Page (</learning-centre>)

Component: [DocumentCard](#)

Data source: [src/data/documents.ts](#)

9 downloadable resources grouped by type:

#	Document	Level	Size
1	2023 A/L Combined Maths Past Paper	A/L	2.4 MB
2	2022 A/L Combined Maths Past Paper	A/L	2.1 MB
3	2023 A/L Physics Past Paper	A/L	3.1 MB
4	2022 O/L Mathematics Past Paper	O/L	1.8 MB
5	Calculus Model Answers Ch. 1–5	A/L	1.2 MB
6	Mechanics Quick Reference Notes	A/L	890 KB
7	Statistics & Probability Revision Notes	A/L	1.5 MB
8	2021 A/L Physics with Full Answers	A/L	4.2 MB
9	O/L Maths — Geometry & Trigonometry	O/L	760 KB

Each card has a download button linked to a static file in </public/files/>.

Contact Page (</contact>)

Components: [PersonalContactCard](#), [InstitutionContactCard](#)

Data source: <src/data/contact.ts>

Teacher card (Mr. Kamal Perera):

- Profile header with name and title
- Clickable phone numbers ([tel](#): links): +94 77 123 4567, +94 71 987 6543
- Clickable email ([mailto](#): link): kamal.perera@tutiolms.lk
- Address: No. 22, Temple Road, Nugegoda, Colombo
- Social buttons: Facebook, YouTube, Instagram
- Qualifications list (4 items)

Institution cards (3 locations):

- Numbered card header
 - Address
 - Phone numbers
 - Operating hours
-

404 Page (</not-found.tsx>)

Auto-displayed by Next.js for any unknown URL. Shows the TutioLMS branding with a "Go to Home" button.

7. Admin Dashboard — Pages & Features

Access the admin panel at: <http://localhost:3000/admin>

Login Page (/admin)

No sidebar — standalone centered card.

Field	Type	Notes
Email	<code>input[type=email]</code>	Required
Password	<code>input[type=password]</code>	Toggle show/hide

Demo credentials:

Role	Email	Password
Super Admin	admin@tutiolms.lk	Admin@1234
Teacher	teacher@tutiolms.lk	Teacher@1234

On successful login → redirects to `/admin/dashboard`. On failure → shows inline error message.

Dashboard (/admin/dashboard) — Both Roles

SUPER_ADMIN view:

- 4 stat cards: Banners count, Institutions count, Teachers count, Materials count
- Recent Banners list (title + active/inactive badge)
- Institutions overview (name + class count)

TEACHER view:

- "My Materials" count card (own uploads only)
 - "My Profile" shortcut card
 - Recent materials list (title + type + level)
-

Banner Management (/admin/banners) — Super Admin Only

Manages the hero carousel images shown on the homepage.

Table columns: Preview image, Title, Status (Active/Inactive), Created Date

Add/Edit Banner form fields:

- Title (text)
- Image URL (text + live preview thumbnail)
- Active toggle (show/hide on carousel)

Actions: Add Banner button, Edit (pencil icon), Delete (trash icon with confirm dialog)

Institution Management (/admin/institutions) — Super Admin Only

Manages the 3 learning centre locations and their class timetables.

Table columns: Institution name, Address, Phone, Class count

Add/Edit Institution form fields:

- Name, Phone (grid row)
 - Full Address
 - Google Maps URL (optional)
 - Dynamic timetable editor:
 - Add Row / Remove Row buttons
 - Per row: Day, Time, Subject, Level (O/L or A/L dropdown)
-

Theme Settings (/admin/theme) — Super Admin Only

Live colour customisation for the entire website.

Primary colour picker:

- HSL colour wheel from `react-colorful`
- Live colour swatch preview
- HSL values displayed (e.g. `HSL(215, 70%, 25%)`)

Accent colour picker:

- Same HSL picker for the amber accent colour

Navigation visibility:

- Toggle switch per nav item (Home, Institutions, Classes, Learning Centre, Contact)
- Controls which links appear in the public Navbar

Buttons:

- **Apply Preview** — immediately injects CSS variables into `document.documentElement.style` so colour changes are visible on the current page
 - **Save Settings** — persists the full settings object to `localStorage("admin_settings")`
-

Teacher Management (/admin/teachers) — Super Admin Only

Manages teacher/admin user records.

Table columns: Avatar, Name, Email, Phone, Subject, Role badge

Add/Edit Teacher form fields:

- Profile image URL (with circular live preview)
- Full Name, Role dropdown (Teacher / Super Admin)
- Email, Phone (grid row)
- Subject (optional)

New teachers are assigned a UUID via `crypto.randomUUID()`.

Learning Centre Manager (/admin/learning-centre) — Teacher Only

Teachers can only see and manage their own uploaded materials.

Table columns: Title, Subject, Level badge, Type badge (PDF/NOTE/VIDEO), Upload Date

Add/Edit Material form fields:

- Title
- Subject, Level (O/L/A/L), Type (PDF/NOTE/VIDEO) — in a 3-column grid
- **Content editor** — textarea with a formatting toolbar:
 - **B** → wraps selected text in *****bold*****
 - *I* → wraps selected text in *_italic_*
 - U → wraps selected text in <u>underline</u>
 - Link → wraps selected text in `[text](url)`
 - Note: Markdown syntax — not a rendered WYSIWYG editor
- File URL / PDF Link (optional)

Profile (/admin/profile) — Both Roles

Each user can update their own contact details.

Form fields:

- Profile image URL (circular live preview)
- Full Name
- Email Address
- Phone Number

On save: updates the `AuthContext` user object and writes back to `localStorage("admin_user")`. Changes persist across page refreshes.

8. Authentication & Role System

How Auth Works (Current — Mock)

```
User enters credentials
    ↓
login() in AuthContext checks against MOCK_CREDENTIALS lookup
    ↓
If match → find AdminUser from MOCK_USERS array
    ↓
Store AdminUser JSON in localStorage("admin_user")
    ↓
Set user state in React context
    ↓
Admin layout redirects to /admin/dashboard
```

On page load: `AuthContext` reads `localStorage("admin_user")` during a `useEffect` and restores the session without requiring re-login.

On logout: clears `localStorage` + state → redirects to `/admin`.

Roles

Role	Value	Access
Super Admin	"SUPER_ADMIN"	All admin pages
Teacher	"TEACHER"	Dashboard, Learning Centre, Profile

ProtectedRoute HOC

Every admin page wraps its content with `<ProtectedRoute allowedRoles={ [...]}>`:

```
// Example – banners page (SUPER_ADMIN only)
export default function BannersPage() {
  return (
    <ProtectedRoute allowedRoles={["SUPER_ADMIN"]}>
      <BannersContent />
    </ProtectedRoute>
  );
}
```

If a TEACHER visits `/admin/banners`, `ProtectedRoute` silently redirects them to `/admin/dashboard`. If nobody is logged in, they are sent to `/admin` (login page).

Admin Layout Redirect Logic

`src/app/admin/layout.tsx` handles three cases on every navigation:

Condition	Action
On <code>/admin</code> (login page) AND authenticated	Redirect to <code>/admin/dashboard</code>
On any <code>/admin/*</code> page AND NOT authenticated	Redirect to <code>/admin</code>
On any <code>/admin/*</code> page AND authenticated	Show sidebar + header shell

9. Data Layer

Current State: Static Files + Local State

All data is currently stored in TypeScript files under `src/data/`. The admin dashboard reads from these on initial render and keeps changes in React `useState` — no network calls.

Public Data Files

File	Contents	Used By
<code>src/data/navigation.ts</code>	5 nav items with labels and hrefs	<code>Navbar.tsx</code> , <code>AdminSidebar</code> , <code>mockSettings</code>
<code>src/data/slides.ts</code>	4 hero carousel slides	<code>HeroCarousel.tsx</code>
<code>src/data/classes.ts</code>	5 categories, 18 total class items	<code>AccordionList.tsx</code>
<code>src/data/institutions.ts</code>	3 institutions with timetable arrays	<code>InstitutionCard.tsx</code> , <code>admin/institutions</code>
<code>src/data/documents.ts</code>	9 downloadable materials	<code>DocumentCard.tsx</code> , <code>mockMaterials</code>
<code>src/data/contact.ts</code>	Teacher + institution contact info	<code>PersonalContactCard.tsx</code> , <code>InstitutionContactCard.tsx</code>

Admin Mock Data Files

File	Contents
<code>src/data/admin/mockUsers.ts</code>	2 credential entries + 2 AdminUser records
<code>src/data/admin/mockBanners.ts</code>	3 banners (2 active, 1 inactive)
<code>src/data/admin/mockTeachers.ts</code>	2 teacher records (mirrors mockUsers)
<code>src/data/admin/mockMaterials.ts</code>	8 learning materials (seeded from documents.ts)
<code>src/data/admin/mockSettings.ts</code>	Default theme HSL values + all 5 nav items visible

Mock Credentials

```
// src/data/admin/mockUsers.ts
export const MOCK_CREDENTIALS = {
  "admin@tutiolms.lk": { password: "Admin@1234",   userId: "user-001" },
  "teacher@tutiolms.lk": { password: "Teacher@1234", userId: "user-002" },
};
```

How to Replace Mock Data with Real API

When the NestJS backend is ready, replace static imports in each admin page:

Before (mock):

```
import { mockBanners } from "@data/admin/mockBanners";
const [banners, setBanners] = useState<Banner[]>(mockBanners);
```

After (real API):

```
const [banners, setBanners] = useState<Banner[]>([]);

useEffect(() => {
  fetch("http://localhost:4000/banners", {
    headers: { Authorization: `Bearer ${token}` },
  })
    .then((r) => r.json())
    .then(setBanners);
}, []);
```

10. Theming & Styling System

CSS Custom Properties (Variables)

Defined in `src/app/globals.css`:

```
:root {
  --color-primary:      215 70% 25%; /* Deep Navy Blue */
  --color-primary-light: 215 70% 38%; /* Hover state */
  --color-primary-dark:  215 70% 15%; /* Footer background */
  --color-accent:        38  95% 53%; /* Amber gold */
}
```

These are HSL values without the `hsl()` wrapper. Tailwind reads them with the `/ <alpha-value>` pattern:

```
// tailwind.config.ts
colors: {
  primary: {
    DEFAULT: "hsl(var(--color-primary) / <alpha-value>)",
    light:    "hsl(var(--color-primary-light) / <alpha-value>)",
    dark:     "hsl(var(--color-primary-dark) / <alpha-value>)",
  },
  accent: {
    DEFAULT: "hsl(var(--color-accent) / <alpha-value>)",
  },
}
```

Using Theme Colours in Components

```
// Solid primary background
<div className="bg-primary">...</div>

// Primary with opacity
```

```
<div className="bg-primary/10">...</div> // 10% opacity navy

// Primary-dark (footer)
<div className="bg-primary-dark">...</div>

// Accent
<span className="text-accent">...</span>
```

Live Theme Updates (Admin)

The Theme Editor writes new HSL values directly to the DOM:

```
document.documentElement.style.setProperty(
  "--color-primary",
  `${h} ${s}% ${l}%`
);
```

This updates all Tailwind colour utilities that reference the variable instantly — no page reload needed.

Admin Dashboard Styling Conventions

Element	Tailwind Classes
Sidebar background	bg-white border-r border-gray-200
Active nav link	bg-primary text-white rounded-lg
Inactive nav link	text-gray-600 hover:bg-gray-100 rounded-lg
Page background	bg-gray-50
Content card	bg-white rounded-xl border border-gray-200
Table header	bg-gray-50 text-xs uppercase text-gray-500 tracking-wider
Form input	border border-gray-300 rounded-lg px-3 py-2 text-sm focus:ring-2 focus:ring-primary/30 focus:border-primary outline-none
Danger button	bg-red-500 hover:bg-red-600 text-white
Active status badge	bg-green-100 text-green-700
Inactive status badge	bg-gray-100 text-gray-500

11. Reusable Component Library

Public UI (src/components/ui/)

Button

```
import Button from "@components/ui/Button";

// Variants: "primary" | "outline" | "ghost"
// Sizes: "sm" | "md" | "lg"
<Button variant="primary" size="md">Enroll Now</Button>
<Button variant="outline" size="sm">Learn More</Button>
```

Card

```
import Card from "@components/ui/Card";

<Card className="p-6">
  Content here
</Card>
```

SectionHeading

```
import SectionHeading from "@components/ui/SectionHeading";

<SectionHeading title="Our Classes" subtitle="Choose the format that works for
you" centered />
```

Admin Shared ([src/components/admin/shared/](#))

DataTable (Generic)

Accepts typed column definitions and typed row data:

```
import DataTable from "@components/admin/shared/DataTable";
import type { Column } from "@types/admin";

const columns: Column<Banner>[] = [
  { key: "title", header: "Title" },
  {
    key: "isActive",
    header: "Status",
    render: (val) => <span>{val ? "Active" : "Inactive"}</span>,
  },
];

<DataTable<Banner>
  columns={columns}
```

```

data={banners}
onEdit={(row) => openEdit(row)}
onDelete={(row) => setDeleteTarget(row)}
emptyMessage="No banners yet."
/>

```

Modal

```

import Modal from "@components/admin/shared/Modal";

<Modal
  isOpen={modalMode !== null}
  onClose={closeModal}
  title="Edit Banner"
  size="md" // "sm" | "md" | "lg"
>
  <BannerForm ... />
</Modal>

```

Closes on: backdrop click, Escape key, or manual `onClose()` call.

ProtectedRoute

```

import ProtectedRoute from "@components/admin/shared/ProtectedRoute";

export default function TeachersPage() {
  return (
    <ProtectedRoute allowedRoles={["SUPER_ADMIN"]}>
      <TeachersContent />
    </ProtectedRoute>
  );
}

```

StatCard

```

import StatCard from "@components/admin/shared/StatCard";
import { BookOpen } from "lucide-react";

<StatCard
  title="Materials"
  value={42}
  icon={BookOpen}
  href="/admin/learning-centre"
  color="primary" // "primary" | "accent" | "green" | "red"
/>

```

PageHeader

```
import PageHeader from "@components/admin/shared/PageHeader";

<PageHeader
  title="Banner Management"
  subtitle="Manage carousel banners shown on the homepage."
  action={<button onClick={openAdd}>Add Banner</button>}
/>
```

12. NestJS Backend Reference

The files in `backend-reference/` provide a complete, production-ready NestJS architecture. They are **excluded from TypeScript compilation** (see `tsconfig.json`) because NestJS packages are not installed in this Next.js project.

RBAC Implementation Pattern

Step 1 — Decorate your route with allowed roles:

```
@Get()
@Roles(Role.SUPER_ADMIN) // Only super admins can GET
findAll() { ... }

@Get('me')
@Roles(Role.TEACHER) // Only teachers can GET their own
findOwn(@Request() req) { ... }
```

Step 2 — Apply guards to the controller:

```
@Controller('banners')
@UseGuards(JwtAuthGuard, RolesGuard) // Auth first, then role check
export class BannersController { ... }
```

Step 3 — JwtAuthGuard populates req.user:

```
// After JWT validation, req.user =
{ id: "uuid", email: "admin@...", role: "SUPER_ADMIN" }
```

Step 4 — RolesGuard reads @Roles() metadata and checks req.user.role.

RBAC Access Matrix

Endpoint	SUPER_ADMIN	TEACHER
GET /banners	All banners	—
POST /banners	Create	—
PATCH /banners/:id	Update any	—
DELETE /banners/:id	Delete any	—
GET /institutions	All	—
POST /institutions	Create	—
PATCH /institutions/:id	Update any	—
DELETE /institutions/:id	Delete any	—
GET /learning-materials	All materials	Own materials only
POST /learning-materials	Create	Create (auto-assigns uploaderId)
PATCH /learning-materials/:id	Update any	Own materials only
DELETE /learning-materials/:id	Delete any	—
GET /site-settings	All settings	—
PUT /site-settings/:key	Update setting	—
POST /auth/login	Public	Public

13. Database Schema (Prisma)

Located at [backend-reference/prisma/schema.prisma](#).

Models

```

User
  id          UUID (PK)
  email       String (unique)
  password    String (bcrypt hash)
  name        String
  phone       String?
  imageUrl    String?
  role        Role enum (SUPER_ADMIN | TEACHER)
  createdAt   DateTime
  updatedAt   DateTime
  materials   LearningMaterial[]

Banner
  id          UUID (PK)
  title       String
  imageUrl    String
  isActive    Boolean (default: true)

```

```

    sortOrder Int (default: 0)
    createdAt DateTime
    updatedAt DateTime

Institution
  id          UUID (PK)
  name        String
  address      String
  phone        String
  mapUrl       String?
  timetable    TimetableRow[]
  createdAt    DateTime
  updatedAt    DateTime

TimetableRow
  id          UUID (PK)
  day          String
  time         String
  subject       String
  level         String ("O/L" | "A/L")
  institution   Institution (FK, cascade delete)
  institutionId UUID

LearningMaterial
  id          UUID (PK)
  title        String
  type         MaterialType enum (PDF | NOTE | VIDEO)
  content       String (Text – long form)
  fileUrl       String?
  subject       String
  level         String ("O/L" | "A/L")
  uploader      User (FK)
  uploaderId    UUID
  createdAt     DateTime
  updatedAt     DateTime

SiteSetting
  key          String (PK – e.g. "primaryHSL", "navItems")
  value         Json
  updatedAt     DateTime

```

Relationships

- `User` → `LearningMaterial[]` (one-to-many: teacher uploads many materials)
- `Institution` → `TimetableRow[]` (one-to-many, cascade delete)

14. API Endpoints Reference

Base URL (when backend is running): <http://localhost:4000>

All endpoints except `POST /auth/login` require:

```
Authorization: Bearer <jwt_token>
```

Authentication

Method	Path	Auth	Body	Response
POST	/auth/login	None	{ email, password }	{ access_token, user }

Banners (SUPER_ADMIN only)

Method	Path	Auth	Body
GET	/banners	SUPER_ADMIN	—
GET	/banners/:id	SUPER_ADMIN	—
POST	/banners	SUPER_ADMIN	{ title, imageUrl, isActive }
PATCH	/banners/:id	SUPER_ADMIN	Partial banner fields
DELETE	/banners/:id	SUPER_ADMIN	—

Institutions (SUPER_ADMIN only)

Method	Path	Auth	Body
GET	/institutions	SUPER_ADMIN	—
GET	/institutions/:id	SUPER_ADMIN	—
POST	/institutions	SUPER_ADMIN	{ name, address, phone, mapUrl, timetable[] }
PATCH	/institutions/:id	SUPER_ADMIN	Partial fields
DELETE	/institutions/:id	SUPER_ADMIN	—

Learning Materials (Mixed roles)

Method	Path	Auth	Notes
GET	/learning-materials	Both	SUPER_ADMIN gets all; TEACHER gets own
GET	/learning-materials/:id	Both	TEACHER restricted to own
POST	/learning-materials	Both	uploaderId auto-set from JWT
PATCH	/learning-materials/:id	Both	TEACHER restricted to own
DELETE	/learning-materials/:id	SUPER_ADMIN	—

Site Settings (SUPER_ADMIN only)

Method	Path	Body
GET	/site-settings	—
GET	/site-settings/:key	—
PUT	/site-settings/:key	{ value: any }

15. Configuration Files

package.json — Scripts & Dependencies

```
{
  "scripts": {
    "dev": "next dev",      // Start dev server (hot reload)
    "build": "next build",  // Production build
    "start": "next start",  // Start production server
    "lint": "next lint"     // Run ESLint
  },
  "dependencies": {
    "next": "14.2.14",
    "react": "^18",
    "react-dom": "^18",
    "lucide-react": "^0.441.0",
    "react-colorful": "^5.6.1"
  }
}
```

tsconfig.json — TypeScript

Key settings:

- "strict": true — full type checking enabled
- "paths": { "@/*": ["/src/*"] } — import alias
- "exclude": ["node_modules", "backend-reference"] — NestJS reference files excluded from Next.js compilation
- "moduleResolution": "bundler" — Next.js 14 bundler mode

tailwind.config.ts — Styling

Key customisations:

- Custom primary colour tokens (DEFAULT, light, dark) reading CSS variables
- Custom accent colour token
- Custom fontFamily.sans reading --font-sans CSS variable
- Custom transitionProperty.max-height for accordion animations

next.config.mjs — Next.js

```
const nextConfig = {
  images: { unoptimized: true }
};
```

`unoptimized: true` allows `<img>` tags with any external URL without domain whitelist configuration — useful for admin image URL inputs.

16. Environment Variables

For the Next.js Frontend

No environment variables are required for the current mock-data setup.

When connecting to a real backend, create `.env.local` in the project root:

```
NEXT_PUBLIC_API_URL=http://localhost:4000
```

Usage in code:

```
const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/banners`);
```

For the NestJS Backend

Create `.env` in the NestJS project root:

```
# PostgreSQL connection string
DATABASE_URL="postgresql://postgres:password@localhost:5432/tutiolms"

# JWT signing secret (minimum 32 characters – keep private)
JWT_SECRET="change-this-to-a-random-secure-string-in-production"

# Token expiry
JWT_EXPIRES_IN="7d"

# API server port
PORT=4000
```

17. Adding New Features

Add a New Public Page

1. Create `src/app/your-page/page.tsx`

2. Export a default React component
3. Add data to `src/data/your-data.ts` if needed
4. Add a nav link to `src/data/navigation.ts`

Add a New Admin Section (SUPER_ADMIN only)

1. Add the nav item to `SUPER_ADMIN_NAV` in `src/components/admin/layout/AdminSidebar.tsx`
2. Add the breadcrumb label to `ROUTE_LABELS` in `AdminHeader.tsx`
3. Create `src/app/admin/your-section/page.tsx`
4. Wrap content with `<ProtectedRoute allowedRoles={["SUPER_ADMIN"]}>`
5. Create `src/data/admin/mockYourData.ts` with seed data
6. Create form component at `src/components/admin/your-section/YourForm.tsx`
7. Add TypeScript interface to `src/types/admin.ts`

Add a New Prisma Model (Backend)

1. Add the model to `backend-reference/prisma/schema.prisma`
2. Run `npx prisma migrate dev --name add-your-model` in the NestJS project
3. Create `src/your-module/your-module.controller.ts` following the banner controller pattern
4. Apply `@UseGuards(JwtAuthGuard, RolesGuard)` and `@Roles(...)` as needed
5. Register the module in `src/app.module.ts`

Change the Site Theme Colours

Option A — Edit CSS variables (permanent): Open `src/app/globals.css` and change the HSL values:

```
:root {
  --color-primary: 260 70% 30%; /* e.g. deep purple */
  --color-accent: 142 70% 45%; /* e.g. green */
}
```

Option B — Admin Theme Editor (session/localStorage): Log in as Super Admin → go to `/admin/theme` → adjust sliders → Save Settings.

Quick Reference

Admin Login

```
URL:      http://localhost:3000/admin
SuperAdmin: admin@tutiolms.lk / Admin@1234
Teacher:  teacher@tutiolms.lk / Teacher@1234
```

Start Frontend

```
cd c:/Users/AVONET/Desktop/lms
npm run dev
# → http://localhost:3000
```

Build for Production

```
npm run build
npm run start
```

Key File Locations

What	Where
Site colours	src/app/globals.css
Navigation items	src/data/navigation.ts
Hero slides	src/data/slides.ts
Class schedules	src/data/classes.ts
Institution data	src/data/institutions.ts
Download documents	src/data/documents.ts
Teacher contact	src/data/contact.ts
Admin credentials	src/data/admin/mockUsers.ts
Admin types	src/types/admin.ts
Auth logic	src/contextts/AuthContext.tsx
DB schema	backend-reference/prisma/schema.prisma
RBAC guard	backend-reference/src/auth/roles.guard.ts